



C socket docs

☰ Tags	OP
# Cap	-99
⚙ Status	Done
🕒 Ultima modifica	@February 15, 2024 11:12 PM

Info e consigli

Generici

Specifici C

Marcatori per tipi di dato

Lettura e scrittura std

Lettura da stdin:

Scrittura su stdout:

Interazione con file

Apertura e chiusura dei file:

Lettura e scrittura di testo:

Lettura e scrittura di byte:

Creazione, eliminazione, spostamento del file:

Posizionamento all'interno del file:

Esecuzione comandi

Funzione `system()`

Utilizzo

Funzioni `fork()`, `exec()`, `wait()`

Utilizzo

Argomenti delle funzioni `exec()`

Comunicazione UDP (Datagram)

Introduzione alle Socket UDP in C

Protocollo Socket Datagram

Funzioni principali:

Comunicazione TCP (Stream)

Introduzione alle Socket TCP in C

Protocollo Socket Stream

Funzioni principali:

Select

Funzione `select()`

Prototipo:

Parametri:

Valore di ritorno:

Descrizione:

Esempio:

String (GPT)

Funzioni di manipolazione delle stringhe

Altre funzioni utili

File (GPT)

Gestione di cartelle e file

File stats (GPT)

Documentazione sull'utilizzo di `stat` in C

Introduzione a `stat`

Struttura `stat`

Utilizzo di `stat`

Descrizione dei campi della struttura `stat`

Info e consigli

Generici

- Fare i controlli per i parametri in ingresso ovunque
- Non usare funzioni
- Non usare struct e/o classi
- Tutte le variabili vanno inizializzate e istanziate al di fuori dei cicli
- Utilizzare i tipi che occupano meno spazio possibile

Specifici C

- Compilazione `gcc nome_file.c -o nome_eseguibile`
-
-

Marcatori per tipi di dato

SPECIFICATORE	USATO PER
%c	un unico carattere
%s	una stringa
%hi	short (con segno)
%hu	short (senza segno)
%Lf	long double
%n	non stampa nulla
%d	un intero decimale (si assume base 10)
%i	un numero intero decimale (rileva automaticamente la base)
%o	un intero ottale (in base 8).
%x	un intero esadecimale (base 16)
%p	un indirizzo (o puntatore)
%f	un numero in virgola mobile per i float
%u	int decimale senza segno
%e	un numero in virgola mobile in notazione scientifica
%E	un numero in virgola mobile in notazione scientifica
%%	il simbolo %

Lettura e scrittura std

Lettura da stdin:

- `int scanf(const char *format, ...);` : Legge l'input da `stdin` secondo il formato specificato.

```
// Esempio standard
int gigi;
scanf("%d", &gigi);
```

```
// Esempio array (str)
char stringa[50];
scanf("%s", stringa);
```

- `char *fgets(char *str, int n, FILE *stream);` : Legge una riga da `stdin` e la memorizza nella stringa `str`.

```
char buffer[100]; // Buffer per memorizzare la riga letta

printf("Inserisci una riga di testo: ");
fgets(buffer, sizeof(buffer), stdin); // Legge una riga da
```

Scrittura su stdout:

- `int printf(const char *format, ...);` : Formatta e stampa output su `stdout` secondo il formato specificato.

```
// Esempio standard
int gigi
printf("Numero: %d", gigi);

// Esempio array (str)
char stringa[50];
printf("%s", stringa);
```

Inoltre, ci sono altre funzioni correlate che possono essere utili:

- **Lettura di un singolo carattere da stdin:**
 - `char getchar(void);` : Legge un singolo carattere da `stdin`.
- **Scrittura di un singolo carattere su stdout:**
 - `char putchar(int c);` : Scrive il carattere `c` su `stdout`.

Interazione con file

Apertura e chiusura dei file:

- `FILE *fopen(const char *filename, const char *mode);` : Apre un file specificato da `filename` in modalità specificata da `mode`.
 - `"r"` : Apre il file in modalità di lettura. Il file deve esistere, altrimenti l'apertura fallisce.
 - `"w"` : Apre il file in modalità di scrittura. Se il file esiste, il suo contenuto viene eliminato. Se il file non esiste, viene creato.
 - `"a"` : Apre il file in modalità di aggiunta (append). I dati vengono aggiunti alla fine del file, senza eliminare il contenuto esistente.
 - `"r+"` : Apre il file in modalità di lettura e scrittura. Il file deve esistere.
 - `"w+"` : Apre il file in modalità di lettura e scrittura, eliminando il contenuto esistente o creando un nuovo file se non esiste.
 - `"a+"` : Apre il file in modalità di lettura e aggiunta (append), consentendo di leggere e aggiungere dati alla fine del file.
- `int fclose(FILE *stream);` : Chiude il file associato al puntatore `stream`.

Lettura e scrittura di testo:

- `int fprintf(FILE *stream, const char *format, ...);` : Scrive nel file `stream` secondo il formato specificato.

```
FILE *file;
int numero = 10;

// Apriamo il file in modalità scrittura
file = fopen("output.txt", "w");

if (file == NULL) {
    printf("Impossibile aprire il file.\n");
    return 1;
}

// Scriviamo su file usando fprintf
```

```
fprintf(file, "Questo è un numero: %d\n", numero);

// Chiudiamo il file
fclose(file);
```

- `int fscanf(FILE *stream, const char *format, ...);` : Legge dal file `stream` secondo il formato specificato.

```
FILE *file;
int numero;

// Apriamo il file in modalità lettura
file = fopen("input.txt", "r");

if (file == NULL) {
    printf("Impossibile aprire il file.\n");
    return 1;
}

// Leggiamo il numero dal file usando fscanf
fscanf(file, "%d", &numero);

// Chiudiamo il file
fclose(file);

// Stampiamo il numero letto
printf("Numero letto dal file: %d\n", numero);
```

- `char *fgets`

```
FILE *file;
char buffer[100];

// Apre il file in modalità lettura
file = fopen("test.txt", "r");
if (file == NULL) {
    perror("Errore nell'apertura del file");
    return 1;
}
```

```

}

// Legge una riga dal file
fgets(buffer, sizeof(buffer), file);
printf("Riga letta dal file: %s", buffer);

```

Letture e scrittura di byte:

- `size_t fread(void *ptr, size_t size, size_t nmemb, FILE *stream);`: Legge `nmemb` elementi di dimensione `size` da `stream` e li memorizza nel buffer puntato da `ptr`.

```

FILE *file;
char buffer[100]; // Buffer per memorizzare i dati letti
size_t elements_read;

// Apertura del file in modalità lettura binaria
file = fopen("data.bin", "rb");

if (file == NULL) {
    printf("Impossibile aprire il file.\n");
    return 1;
}

// Legge dati dal file fino alla fine del file
while ((elements_read = fread(buffer, sizeof(char), sizeof(buffer), file)) > 0) {
    // Stampa i dati letti
    printf("Numero di elementi letti: %zu\n", elements_read);
    printf("Dati letti: %s\n", buffer);
}

// Chiude il file
fclose(file);

```

- `size_t fwrite(const void *ptr, size_t size, size_t nmemb, FILE *stream);`: Scrive `nmemb` elementi di dimensione `size` nel file `stream` dal buffer puntato da `ptr`.

```

FILE *file;
char buffer[] = "Hello, world!"; // Dati da scrivere nel file
size_t elements_written;

// Apertura del file in modalità scrittura binaria
file = fopen("output.txt", "wb");

if (file == NULL) {
    printf("Impossibile aprire il file.\n");
    return 1;
}

// Scrive i dati nel file usando fwrite
elements_written = fwrite(buffer, sizeof(char), sizeof(buffer), file);

// Chiude il file
fclose(file);

// Stampa il numero di elementi scritti
printf("Numero di elementi scritti: %zu\n", elements_written);

```

Creazione, eliminazione, spostamento del file:

- `int remove(const char *filename);` : Elimina il file specificato da `filename` .

```

char filename[] = "file_da_rimuovere.txt";

// Tentativo di rimuovere il file
if (remove(filename) == 0) {
    printf("Il file è stato rimosso con successo.\n");
} else {
    printf("Impossibile rimuovere il file.\n");
}

```

- `int rename(const char *oldname, const char *newname);` : Rinomina (sposta) un file da `oldname` a `newname` .


```

char oldname[] = "file_da_rinominare.txt";
char newname[] = "nuovo_nome.txt";

// Tentativo di rinominare il file
if (rename(oldname, newname) == 0) {
    printf("Il file è stato rinominato con successo.\n");
} else {
    printf("Impossibile rinominare il file.\n");
}

```

- `int feof(FILE *stream);` : Restituisce vero se il fine del file è stato raggiunto per il file `stream`.

```

FILE *file;
char c;

// Apriamo il file in modalità lettura
file = fopen("test.txt", "r");

if (file == NULL) {
    printf("Impossibile aprire il file.\n");
    return 1;
}

// Leggiamo il file carattere per carattere
while ((c = fgetc(file)) != EOF) {
    // Stampa il carattere letto
    printf("%c", c);
}

// Verifica se abbiamo raggiunto la fine del file
if (feof(file)) {
    printf("\nFine del file raggiunta.\n");
} else {
    printf("\nFine del file non raggiunta.\n");
}

```

```
// Chiudiamo il file
fclose(file);
```

Posizionamento all'interno del file:

- `int fseek(FILE *stream, long offset, int whence);` : Imposta il cursore di posizionamento nel file `stream`.
 - `SEEK_SET` : l'offset è calcolato rispetto all'inizio del file.
 - `SEEK_CUR` : l'offset è calcolato rispetto alla posizione corrente del cursore di posizione.
 - `SEEK_END` : l'offset è calcolato rispetto alla fine del file.

```
FILE *file;

// Apriamo il file in modalità lettura
file = fopen("test.txt", "r");

if (file == NULL) {
    printf("Impossibile aprire il file.\n");
    return 1;
}

// Spostiamoci all'inizio del file
if (fseek(file, 0, SEEK_SET) != 0) {
    printf("Impossibile posizionarsi all'inizio del file.\n");
    fclose(file);
    return 1;
}

// Ora siamo posizionati all'inizio del file
printf("Siamo posizionati all'inizio del file.\n");

// Chiudiamo il file
fclose(file);
```

- `long ftell(FILE *stream);` : Restituisce la posizione corrente nel file `stream`.

```

FILE *file;
long int position;

// Apriamo il file in modalità lettura
file = fopen("test.txt", "r");

if (file == NULL) {
    printf("Impossibile aprire il file.\n");
    return 1;
}

// Ottieni la posizione corrente all'interno del file
position = ftell(file);

if (position == -1L) {
    printf("Errore nell'ottenere la posizione corrente del
} else {
    printf("La posizione corrente nel file è: %ld\n", posi
}

// Chiudiamo il file
fclose(file);

```

- `void rewind(FILE *stream);` : Riavvolge il file `stream` all'inizio.

```

FILE *file;
char c;

// Apriamo il file in modalità lettura
file = fopen("test.txt", "r");

if (file == NULL) {
    printf("Impossibile aprire il file.\n");
    return 1;
}

// Leggiamo il file carattere per carattere
while ((c = fgetc(file)) != EOF) {

```

```
// Stampa il carattere letto
printf("%c", c);
}

// Riavvolgiamo il cursore di posizione all'inizio del file
rewind(file);

// Leggiamo nuovamente il file e stampiamo il suo contenuto
printf("\nContenuto del file dopo il riavvolgimento:\n");
while ((c = fgetc(file)) != EOF) {
    // Stampa il carattere letto
    printf("%c", c);
}

// Chiudiamo il file
fclose(file);
```

Esecuzione comandi

Funzione `system()`

La funzione `system()` permette di eseguire un comando di shell da un programma C.

Utilizzo

```
#include <stdlib.h>

int main() {
    int status = system("ls -l");
    return 0;
}
```

```
}
```

Il comando `"ls -l"` verrà eseguito e il risultato sarà stampato sul terminale.

Funzioni `fork()`, `exec()`, `wait()`

Queste funzioni offrono un controllo più granulare sul processo figlio e sul suo ambiente.

Utilizzo

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main() {
    pid_t pid = fork();

    if (pid == 0) { // processo figlio
        execl("/bin/ls", "ls", "-l", (char *)NULL);
    } else if (pid > 0) { // processo genitore
        wait(NULL);
        printf("Esecuzione del comando ls completata.\n");
    } else {
        perror("Errore durante la fork");
        exit(EXIT_FAILURE);
    }

    return 0;
}
```

In questo esempio, il programma crea un processo figlio usando `fork()`, quindi il processo figlio esegue il comando `ls -l` usando `execl()`. Il processo genitore attende il termine dell'esecuzione del processo figlio con `wait()`.

Argomenti delle funzioni `exec()`

Tutte le varianti della funzione `exec()` accettano argomenti per specificare il programma da eseguire e i suoi eventuali argomenti.

- `execl(const char *path, const char *arg0, ..., (char *)NULL)` : Accetta una lista di argomenti separati da virgole. Il primo argomento `arg0` è il nome del programma.
- `execv(const char *path, char *const argv[])` : Accetta un array di puntatori a stringhe `argv[]` che rappresentano gli argomenti del programma.
- `execle(const char *path, const char *arg0, ..., char *const envp[])` : Simile ad `execl()`, ma consente di specificare anche le variabili d'ambiente.
- `execvp(const char *file, char *const argv[])` : Simile a `execv()`, ma cerca l'eseguibile specificato nei percorsi di ricerca definiti dalla variabile d'ambiente `PATH`.
- `execvpe(const char *file, char *const argv[], char *const envp[])` : Simile a `execvp()`, ma consente di specificare anche le variabili d'ambiente.

Ogni funzione `exec()` sovrascrive l'immagine del processo corrente con il programma specificato.

Comunicazione UDP (Datagram)

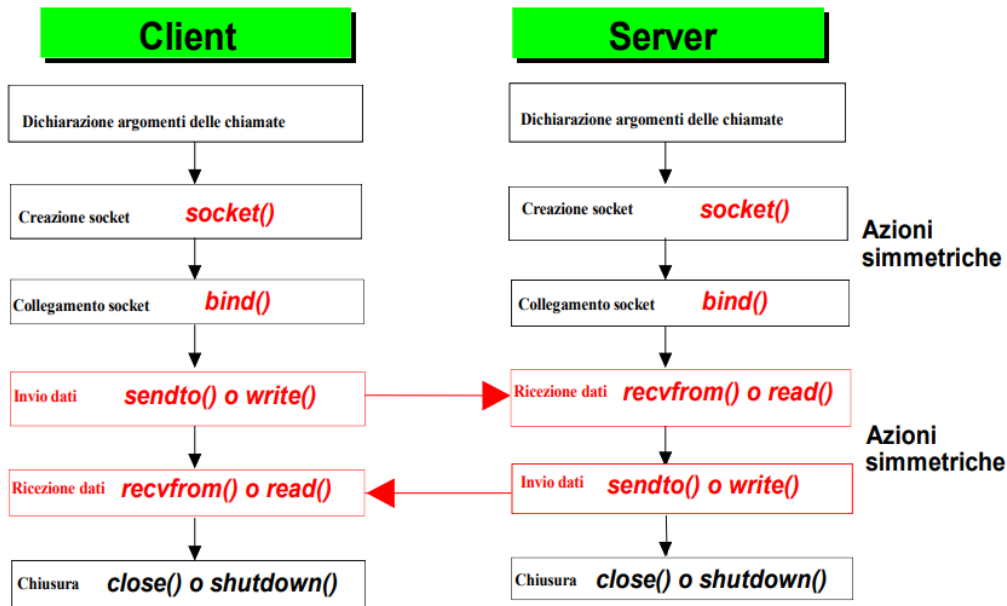
Introduzione alle Socket UDP in C

Le socket UDP (User Datagram Protocol) in C consentono la comunicazione tramite datagrammi senza connessione su una rete. Le UDP sono più leggere e più veloci delle socket TCP, ma offrono meno garanzie di consegna dei pacchetti.

▼ Protocollo Socket Datagram

PROTOCOLLO SOCKET DATAGRAM

Le **socket datagram** sono usate con un **protocollo** che si basa sulla sequenza di **primitive** qui sotto (alcune opzionali, vedi quali?)



Socket in C 25

Funzioni principali:

1. **socket():** Crea un endpoint per la comunicazione e restituisce un file descriptor che si riferisce al socket.

```
int socket(int domain, int type, int protocol);  
  
// Esempio pratico  
int sockfd = socket(AF_INET, SOCK_DGRAM, 0);
```

- **domain**: Specifica il dominio dell'indirizzo.
- **type**: Specifica il tipo di socket (in questo caso, **SOCK_DGRAM** per UDP).
- **protocol**: Specifica il protocollo da utilizzare, generalmente impostato a 0 per il protocollo predefinito.

2. **bind():** Associa un indirizzo locale a un socket.

```
int bind(int sockfd, const struct sockaddr *addr, socklen_t addrlen);
```

- `sockfd` : File descriptor del socket.
- `addr` : Puntatore a una struttura `sockaddr` che contiene l'indirizzo da associare al socket.
- `addrlen` : Lunghezza dell'indirizzo.

3. **sendto()**: Invia un messaggio tramite un socket UDP.

```
ssize_t sendto(int sockfd, const void *buf, size_t len,
int flags, const struct sockaddr *dest_addr, socklen_t addrlen);
```

- `sockfd` : File descriptor del socket.
- `buf` : Buffer contenente i dati da inviare.
- `len` : Lunghezza dei dati in bytes.
- `flags` : Opzioni aggiuntive.
- `dest_addr` : Indirizzo del destinatario.
- `addrlen` : Lunghezza dell'indirizzo del destinatario.

4. **recvfrom()**: Riceve un messaggio tramite un socket UDP.

```
ssize_t recvfrom(int sockfd, void *buf, size_t len, int
flags, struct sockaddr *src_addr, socklen_t *addrlen);
```

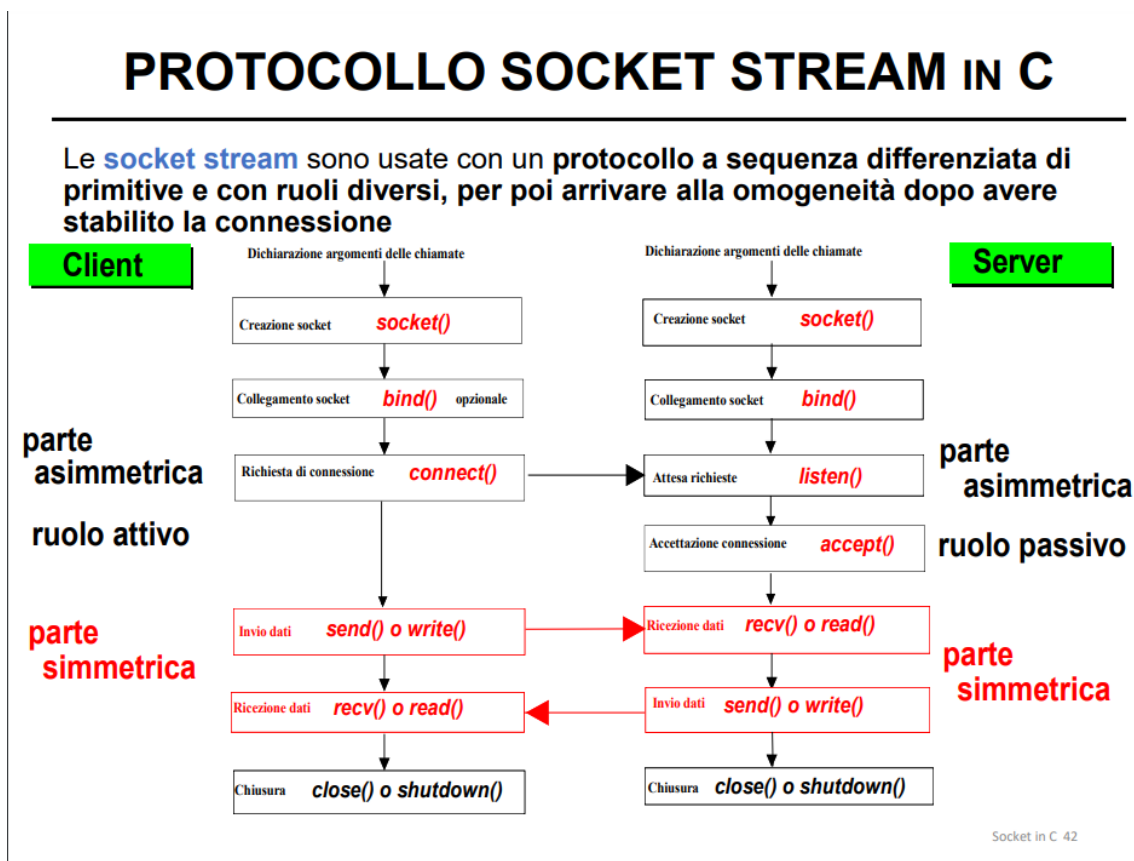
- `sockfd` : File descriptor del socket.
- `buf` : Buffer in cui memorizzare i dati ricevuti.
- `len` : Lunghezza massima dei dati in bytes.
- `flags` : Opzioni aggiuntive.
- `src_addr` : Indirizzo del mittente.
- `addrlen` : Puntatore alla lunghezza dell'indirizzo del mittente.

Comunicazione TCP (Stream)

Introduzione alle Socket TCP in C

Le socket TCP (Transmission Control Protocol) in C consentono la comunicazione affidabile e orientata alla connessione su una rete. A differenza delle socket UDP, le socket TCP garantiscono l'ordine e l'integrità dei dati trasmessi.

▼ Protocollo Socket Stream



Funzioni principali:

1. **socket()**: Crea un endpoint per la comunicazione e restituisce un file descriptor che si riferisce al socket.

```
int socket(int domain, int type, int protocol);
```

- **domain**: Specifica il dominio dell'indirizzo.
- **type**: Specifica il tipo di socket (in questo caso, **SOCK_STREAM** per TCP).

- `protocol` : Specifica il protocollo da utilizzare, generalmente impostato a 0 per il protocollo predefinito.

2. **bind()**: Associa un indirizzo locale a un socket.

```
int bind(int sockfd, const struct sockaddr *addr, socklen_t addrlen);
```

- `sockfd` : File descriptor del socket.
- `addr` : Puntatore a una struttura `sockaddr` che contiene l'indirizzo da associare al socket.
- `addrlen` : Lunghezza dell'indirizzo.

3. **listen()**: Mette il socket in modalità passiva, in attesa di connessioni in ingresso.

```
int listen(int sockfd, int backlog);
```

- `sockfd` : File descriptor del socket.
- `backlog` : Numero massimo di connessioni pendenti che possono essere accettate.

4. **accept()**: Accetta una connessione in arrivo e crea un nuovo socket per la comunicazione.

```
int accept(int sockfd, struct sockaddr *addr, socklen_t *addrlen);
```

- `sockfd` : File descriptor del socket in ascolto.
- `addr` : Puntatore a una struttura `sockaddr` che conterrà l'indirizzo del client.
- `addrlen` : Puntatore alla lunghezza dell'indirizzo.

5. **connect()**: Inizia una connessione a un indirizzo specificato.

```
int connect(int sockfd, const struct sockaddr *addr, socklen_t addrlen);
```

- `sockfd` : File descriptor del socket.
- `addr` : Puntatore a una struttura `sockaddr` che contiene l'indirizzo a cui connettersi.
- `addrlen` : Lunghezza dell'indirizzo.

6. **send()**: Invia dati tramite una connessione socket.

```
int send(int sockfd, const void *buf, size_t len, int flags);
```

- `sockfd` : File descriptor del socket.
- `buf` : Buffer contenente i dati da inviare.
- `len` : Lunghezza dei dati in bytes.
- `flags` : Opzioni aggiuntive.

7. **recv()**: Riceve dati tramite una connessione socket.

```
int recv(int sockfd, void *buf, size_t len, int flags);
```

- `sockfd` : File descriptor del socket.
- `buf` : Buffer in cui memorizzare i dati ricevuti.
- `len` : Lunghezza massima dei dati in bytes.
- `flags` : Opzioni aggiuntive.
 - `0` normale
 - `MSG_OOB` messaggio out-of-band (solo send)
 - `MSG_PEEK` lettura non distruttiva dello stream (solo recv)

8. **write()**: Scrive dati su una connessione socket.

```
int write(int sockfd, const void *buf, size_t count);
```

- `sockfd` : File descriptor del socket.
- `buf` : Buffer contenente i dati da scrivere.
- `count` : Numero di bytes da scrivere.

La funzione `write()` scrive `count` bytes di dati dal buffer `buf` nel socket identificato da `sockfd`. Restituisce il numero di bytes scritti in caso di successo e -1 in caso di errore, impostando `errno` di conseguenza. È importante notare che `write()` può scrivere meno byte di quanti richiesti. In tal caso, è necessario controllare il valore restituito per gestire la scrittura dei dati rimanenti.

9. **read()**: Legge dati da una connessione socket.

```
int read(int sockfd, void *buf, size_t count);
```

- `sockfd`: File descriptor del socket.
- `buf`: Buffer in cui memorizzare i dati letti.
- `count`: Numero massimo di bytes da leggere.

La funzione `read()` tenta di leggere fino a `count` bytes di dati dal socket identificato da `sockfd` e li memorizza nel buffer `buf`. Restituisce il numero di bytes letti in caso di successo e -1 in caso di errore, impostando `errno` di conseguenza. Se non ci sono dati disponibili da leggere, `read()` può bloccare il processo finché non arrivano dati o finché non viene interrotta la lettura. È importante gestire correttamente il valore restituito per determinare quanti bytes sono stati effettivamente letti e trattare eventuali errori di lettura.

Select

Funzione `select()`

La funzione `select()` in C consente di monitorare più file descriptor contemporaneamente per eventi di I/O, come la lettura o la scrittura, senza bloccare il programma. È spesso utilizzata in applicazioni di rete per gestire più connessioni in modo efficiente e concorrente.

Prototipo:

```
int select(int nfd, fd_set *readfds, fd_set *writefds, fd_set *exceptfds, struct timeval *timeout);
```

- `nfd` : Il valore massimo del file descriptor in tutti i set più uno.
- `readfds` : Puntatore al set di file descriptor da monitorare per la lettura.
- `writefds` : Puntatore al set di file descriptor da monitorare per la scrittura.
- `exceptfds` : Puntatore al set di file descriptor da monitorare per le eccezioni.
- `timeout` : Puntatore a una struct timeval che specifica il tempo massimo che `select()` aspetterà per un evento. Può essere impostato su `NULL` per un'attesa infinita.

Parametri:

- `nfd` : Numero intero non negativo che indica il file descriptor più alto in tutti i set più uno.
- `readfds` : Set di file descriptor da monitorare per la lettura.
- `writefds` : Set di file descriptor da monitorare per la scrittura.
- `exceptfds` : Set di file descriptor da monitorare per le eccezioni.
- `timeout` : Tempo massimo di attesa per un evento. Può essere impostato su `NULL` per un'attesa infinita o su una struct timeval che specifica una durata massima di attesa.

Valore di ritorno:

La funzione `select()` restituisce il numero totale di file descriptor pronti per l'I/O, o -1 in caso di errore, impostando `errno` di conseguenza.

Descrizione:

La funzione `select()` monitora i file descriptor specificati nei set `readfds`, `writefds` e `exceptfds` per eventi di I/O. Se uno o più file descriptor diventano pronti per l'I/O prima che scada il timeout specificato, `select()` ritorna il numero di file descriptor pronti e i set sono aggiornati di conseguenza. Se scade il timeout, `select()` ritorna 0. Se si verifica un errore, `select()` ritorna -1 e imposta `errno`.

Esempio:

```
#include <sys/time.h>
#include <sys/types.h>
#include <unistd.h>
#include <stdio.h>

int main() {
    fd_set readfds;
    struct timeval timeout;
    int result;
    int socket_descriptor;

    // Inizializza il set di file descriptor
    FD_ZERO(&readfds);
    FD_SET(socket_descriptor, &readfds);

    // Imposta il timeout a 5 secondi
    timeout.tv_sec = 5;
    timeout.tv_usec = 0;

    // Esegue la select
    result = select(socket_descriptor + 1, &readfds, NULL,
    NULL, &timeout);

    // Gestisce i risultati
    if (result < 0) {
        perror("select() error");
    } else if (result == 0) {
        printf("Timeout\\n");
    } else {
        if (FD_ISSET(socket_descriptor, &readfds)) {
            // Il socket è pronto per la lettura
            // E' possibile utilizzare recv() o altre funzi
            oni di lettura del socket qui
        }
    }
}
```

```
    return 0;
}
```

La funzione `select()` è una potente primitiva di I/O multiplexing che consente a un programma di gestire efficacemente molteplici operazioni di I/O in modo concorrente, migliorando le prestazioni e la reattività dell'applicazione.

String (GPT)

Ecco una documentazione completa delle funzioni sulle stringhe in C, che includono tutte le funzioni standard definite nell'header `<string.h>` e altre funzioni utili:

Funzioni di manipolazione delle stringhe

1. `strlen()`:

- Prototipo: `size_t strlen(const char *str);`
- Descrizione: Restituisce la lunghezza della stringa `str`, escludendo il carattere terminatore nullo `'\0'`.

2. `strcpy()`:

- Prototipo: `char *strcpy(char *dest, const char *src);`
- Descrizione: Copia la stringa `src` nella stringa `dest`, inclusi il terminatore nullo `'\0'`.

3. `strncpy()`:

- Prototipo: `char *strncpy(char *dest, const char *src, size_t n);`
- Descrizione: Copia al massimo `n` caratteri dalla stringa `src` nella stringa `dest`. Se la lunghezza di `src` è inferiore a `n`, il resto di `dest` viene riempito con caratteri nullo `'\0'`.

4. `strcat()`:

- Prototipo: `char *strcat(char *dest, const char *src);`

- Descrizione: Concatena la stringa `src` alla fine della stringa `dest` e aggiunge un terminatore nullo `'\\0'`.

5. **strncat():**

- Prototipo: `char *strncat(char *dest, const char *src, size_t n);`
- Descrizione: Concatena al massimo `n` caratteri dalla stringa `src` alla fine della stringa `dest`. Il risultato sarà sempre terminato da `'\\0'`.

6. **strcmp():**

- Prototipo: `int strcmp(const char *str1, const char *str2);`
- Descrizione: Confronta le stringhe `str1` e `str2`. Restituisce un valore intero negativo se `str1` è meno di `str2`, 0 se sono uguali, e un valore intero positivo se `str1` è maggiore di `str2`.

7. **strncmp():**

- Prototipo: `int strncmp(const char *str1, const char *str2, size_t n);`
- Descrizione: Confronta al massimo `n` caratteri delle stringhe `str1` e `str2`. Restituisce un valore intero negativo se `str1` è meno di `str2`, 0 se sono uguali, e un valore intero positivo se `str1` è maggiore di `str2`.

8. **strstr():**

- Prototipo: `char *strstr(const char *haystack, const char *needle);`
- Descrizione: Restituisce un puntatore alla prima occorrenza di `needle` all'interno di `haystack`, o NULL se `needle` non è trovato.

9. **strchr():**

- Prototipo: `char *strchr(const char *str, int c);`
- Descrizione: Restituisce un puntatore alla prima occorrenza del carattere `c` nella stringa `str`, o NULL se il carattere non è trovato.

10. **strrchr():**

- Prototipo: `char *strrchr(const char *str, int c);`
- Descrizione: Restituisce un puntatore all'ultima occorrenza del carattere `c` nella stringa `str`, o NULL se il carattere non è trovato.

11. **strstr():**

- Prototipo: `char *strstr(const char *haystack, const char *needle);`

- Descrizione: Restituisce un puntatore alla prima occorrenza di `needle` all'interno di `haystack`, o NULL se `needle` non è trovato.

12. `strtok()`:

- Prototipo: `char *strtok(char *str, const char *delim);`
- Descrizione: Restituisce un puntatore al primo token trovato nella stringa `str`, dove i token sono delimitati dalla stringa `delim`. La funzione mantiene lo stato tra le chiamate successive per continuare a estrarre i token successivi dalla stessa stringa.

Altre funzioni utili

1. `memset()`:

- Prototipo: `void *memset(void *ptr, int value, size_t num);`
- Descrizione: Imposta i primi `num` byte dell'area di memoria puntata da `ptr` con il valore specificato `value`.

2. `memcpy()`:

- Prototipo: `void *memcpy(void *dest, const void *src, size_t n);`
- Descrizione: Copia `n` byte dalla memoria di origine `src` all'area di memoria di destinazione `dest`.

3. `memmove()`:

- Prototipo: `void *memmove(void *dest, const void *src, size_t n);`
- Descrizione: Copia `n` byte dalla memoria di origine `src` all'area di memoria di destinazione `dest`, gestendo sovrapposizioni correttamente.

Queste sono tutte le principali funzioni per la manipolazione delle stringhe in C, insieme ad alcune funzioni aggiuntive per la manipolazione della memoria. È possibile trovare molte altre funzioni utili nella libreria standard C, ma queste sono tra le più comuni e utilizzate.

File (GPT)

Ecco gli aggiornamenti alla documentazione per includere lo scorrere di tutti i file di una cartella e la verifica dell'esistenza di un file o di una cartella:

Gestione di cartelle e file

1. Creazione di una cartella:

- Funzione: `int mkdir(const char *path, mode_t mode);`
- Descrizione: Crea una nuova cartella con il percorso specificato.

2. Creazione di un file:

- Funzione: `int creat(const char *path, mode_t mode);`
- Descrizione: Crea un nuovo file con il percorso specificato.

3. Apertura di un file esistente:

- Funzione: `int open(const char *path, int flags, mode_t mode);`
- Descrizione: Apre un file esistente con il percorso specificato.

4. Lettura da un file:

- Funzione: `ssize_t read(int fd, void *buf, size_t count);`
- Descrizione: Legge fino a `count` byte dal file descriptor `fd` nel buffer `buf`.

5. Scrittura su un file:

- Funzione: `ssize_t write(int fd, const void *buf, size_t count);`
- Descrizione: Scrive `count` byte dal buffer `buf` nel file descriptor `fd`.

6. Chiusura di un file:

- Funzione: `int close(int fd);`
- Descrizione: Chiude il file descriptor specificato.

7. Rimozione di un file o di una cartella:

- Funzione: `int remove(const char *path);`
- Descrizione: Rimuove il file o la cartella specificata dal disco.

8. Rinomina un file o una cartella:

- Funzione: `int rename(const char *oldpath, const char *newpath);`
- Descrizione: Rinomina il file o la cartella `oldpath` con il nome `newpath`.

9. Spostamento di un file o una cartella:

- Funzione: `int rename(const char *oldpath, const char *newpath);`
- Descrizione: Sposta il file o la cartella `oldpath` nella posizione `newpath`.

10. Scorrimento di tutti i file di una cartella:

- Funzione: `struct dirent *readdir(DIR *dir);`
- Descrizione: Legge la successiva voce all'interno della directory specificata.
- Esempio di utilizzo:

```
DIR *directory = opendir("/path/to/directory");
if (directory != NULL) {
    struct dirent *entry;
    while ((entry = readdir(directory)) != NULL) {
        printf("Nome file: %s\\n", entry->d_name);
    }
    closedir(directory);
} else {
    printf("Errore durante l'apertura della director
y.\\n");
}
```

11. Verifica se un file o una cartella esiste:

- Funzione: `int access(const char *pathname, int mode);`
- Descrizione: Verifica l'accesso al file o alla cartella specificata.
- Esempio di utilizzo:

```
if (access("/path/to/file_or_directory", F_OK) != -1)
{
    printf("Il file o la cartella esiste.\\n");
} else {
    printf("Il file o la cartella non esiste.\\n");
}
```

Queste sono alcune delle funzioni principali per la gestione di file e cartelle in C. Puoi trovare molte altre funzioni e dettagli nelle rispettive documentazioni di

sistema.

File stats (GPT)

Documentazione sull'utilizzo di `stat` in C

La funzione `stat` in C consente di ottenere informazioni dettagliate su un file specificato dal percorso. Questa documentazione fornirà una panoramica sull'utilizzo di `stat` e spiegherà i dati che è possibile ottenere utilizzando questa funzione.

Introduzione a `stat`

La funzione `stat` è definita nell'header file `<sys/stat.h>`. La sua firma è la seguente:

```
int stat(const char *pathname, struct stat *statbuf);
```

Dove:

- `pathname` è il percorso del file del quale si desidera ottenere le informazioni.
- `statbuf` è un puntatore a una struttura `stat` che conterrà le informazioni ottenute.

Struttura `stat`

La struttura `stat` contiene una serie di campi che forniscono informazioni dettagliate sul file specificato. La sua definizione è la seguente:

```
struct stat {
    dev_t      st_dev;          /* ID del dispositivo contene
nte il file */
    ino_t      st_ino;          /* Numero inode */
    mode_t     st_mode;         /* Modalità di file */
```

```

nlink_t    st_nlink;        /* Numero di hard links */
uid_t      st_uid;          /* ID del proprietario */
gid_t      st_gid;          /* ID del gruppo */
dev_t      st_rdev;         /* Tipo di dispositivo (se il
file è un dispositivo speciale) */
off_t      st_size;         /* Dimensione totale in byte
*/
blksize_t  st_blksize;      /* Dimensione del blocco di
I/O */
blkcnt_t   st_blocks;       /* Numero di blocchi assegnat
i */
time_t     st_atime;        /* Ultimo accesso */
time_t     st_mtime;        /* Ultima modifica */
time_t     st_ctime;        /* Ultimo cambiamento dello s
tato */
};

```

Utilizzo di `stat`

Di seguito viene mostrato un esempio di come utilizzare la funzione `stat` per ottenere informazioni su un file:

```

#include <stdio.h>
#include <sys/stat.h>

int main() {
    const char *filename = "esempio.txt";
    struct stat file_stat;

    // Utilizzare stat per ottenere le informazioni sul fil
    e
    if (stat(filename, &file_stat) == 0) {
        // Stampare le informazioni ottenute
        printf("Informazioni su %s:\n", filename);
        printf("Dimensione: %ld bytes\n", file_stat.st_siz
    e);
        printf("Ultimo accesso: %ld\n", file_stat.st_atim
    e);
    }
}

```

```

        printf("Ultima modifica: %ld\\n", file_stat.st_mtim
e);
        printf("Ultimo cambiamento dello stato: %ld\\n", fi
le_stat.st_ctime);
        // Altri campi della struttura stat possono essere
stampati a seconda delle necessità
    } else {
        perror("stat");
    }

    return 0;
}

```

Descrizione dei campi della struttura **stat**

1. **st_dev** : ID del dispositivo contenente il file.
2. **st_ino** : Numero inode del file.
3. **st_mode** : Modalità di file (permessi di accesso e tipo di file).
4. **st_nlink** : Numero di hard links al file.
5. **st_uid** : ID del proprietario del file.
6. **st_gid** : ID del gruppo del file.
7. **st_rdev** : Tipo di dispositivo (se il file è un dispositivo speciale).
8. **st_size** : Dimensione totale del file in byte.
9. **st_blksize** : Dimensione del blocco di I/O per il file.
10. **st_blocks** : Numero di blocchi assegnati al file.
11. **st_atime** : Tempo dell'ultimo accesso al file.
12. **st_mtime** : Tempo dell'ultima modifica al file.
13. **st_ctime** : Tempo dell'ultimo cambiamento dello stato del file.

Questi campi forniscono informazioni utili sul file che possono essere utilizzate per vari scopi, come il monitoraggio delle modifiche, il calcolo della dimensione del file e altro ancora.